

```

$ kubectl exec -n yb-demo-asia-southeast1-a -it yb-tserver-0 -- ysqlsh -h yb-tserver
.yb-tservers.yb-demo-asia-southeast1-a
Defaulting container name to yb-tserver.
Use 'kubectl describe pod/yb-tserver-0 -n yb-demo-asia-southeast1-a' to see all of the containers in this pod.
ysqlsh (11.2-YB-2.1.6.0-b0)
Type "help" for help.

yugabyte=# CREATE DATABASE TEST30;
CREATE DATABASE
yugabyte=# \l

              List of databases
  Name      | Owner   | Encoding | Collate |  Ctype  | Access privileges
-----+-----+-----+-----+-----+-----
 test20     | yugabyte | UTF8     | C       | en_US.UTF-8 |
 test30     | yugabyte | UTF8     | C       | en_US.UTF-8 |
(2 rows)

yugabyte=# \c test30;
You are now connected to database "test30" as user "yugabyte".
test30=# CREATE TABLE account(
test30(# userid serial PRIMARY KEY,
test30(# firstname VARCHAR (50) UNIQUE NOT NULL,
test30(# lastname VARCHAR (50) UNIQUE NOT NULL,
test30(# password VARCHAR (10) NOT NULL,
test30(# emailid VARCHAR (100) UNIQUE NOT NULL,
test30(# created_on TIMESTAMP NOT NULL
test30(# );
CREATE TABLE
test30=# SELECT * FROM ACCOUNT;
  userid | firstname | lastname | password | emailid | created_on
-----+-----+-----+-----+-----+-----
(0 rows)

test30=# INSERT INTO account(userid, firstname, lastname, password, emailid, created_on)
test30-# VALUES ('1', 'john', 'mathew', 'john1', 'john.mathew@coy.com', now());
INSERT 0 1
test30=#
test30=# INSERT INTO account(userid, firstname, lastname, password, emailid, created_on)
test30-# VALUES ('2', 'sam', 'antony', 'sam2', 'sam.antony@coy.com', now());
INSERT 0 1
test30=#
test30=# INSERT INTO account(userid, firstname, lastname, password, emailid, created_on)
test30-# VALUES ('3', 'imran', 'arian', 'imran3', 'imran.arian@coy.com', now());
INSERT 0 1
test30=#
test30=# INSERT INTO account(userid, firstname, lastname, password, emailid, created_on)
test30-# VALUES ('4', 'alexi', 'kalikov', 'alexil', 'alexi.kalikov@coy.com', now());
INSERT 0 1
test30=# SELECT * FROM ACCOUNT;
  userid | firstname | lastname | password | emailid | created_on
-----+-----+-----+-----+-----+-----
      1 | john     | mathew  | john1    | john.mathew@coy.com | 2020-07-06 05:55:07.586781
      4 | alexi    | kalikov | alexil   | alexi.kalikov@coy.com | 2020-07-06 05:55:09.529932
      2 | sam      | antony  | sam2     | sam.antony@coy.com | 2020-07-06 05:55:07.637664
      3 | imran    | arian   | imran3   | imran.arian@coy.com | 2020-07-06 05:55:07.647246
(4 rows)

```

Figure 1: Screenshot of YSQL execution

can remain fully compatible with PostgreSQL by using a client interface called YSQL, which is a distributed SQL API and is compatible with PostgreSQL. Figure 2 shows standard DDL (data definition), DQL (data query) and DML (data manipulation) operations done using YSQL commands in a Yugabyte database.

We can use YCQL, which is a semi-relational API built for high performance and massive scale (similar to Hbase for the NoSQL Hadoop interface), with its roots in Cassandra Query Language (CQL). This makes YugabyteDB suitable for distributed and NoSQL capabilities. YSQL provides all the following standard facilities of a transaction database system:

- Transaction isolation
- Explicit row locks during critical operations
- Single-row transactions in business processing
- Distributed transactions in batch processing
- Transaction IO path details in enrichment
- Sharding facilities, in which a larger distributed database is logically partitioned into smaller data shards, making it easier to manage the database and faster to query during transaction processing

YSQL also provides tablet splitting, co-located tables in distributed instances, the default replication mechanism to enable high availability or real-time data across nodes, cluster replication

to enable data safety, read replicas for higher performance across nodes, persistence in transaction handling, as well as higher performance in both relational and distributed data handling.

YugaByteDB Cloud instance

The YugaByteDB Cloud instance can be easily created with the pre-defined templates available in Terraform and AWS Cloud formation scripts, as given below.

The following GNU command in Windows or Linux environments downloads the AWS CloudFormation template:

```
wget https://raw.githubusercontent.com/yugabyte/aws-cloudformation/master/yugabyte_cloudformation.yaml
```